

# Lecture 15

## 1. Traditional Routing (HTML/CSS/JS)

In the early days, routing was **Server-Side**. Every time you clicked a link (`<a href="/about.html">`), the browser sent a request to the server, and the server sent back a **completely new HTML file**.

- **The Experience:** The screen goes white for a split second, the "refresh" icon spins, and all JavaScript state is lost.
- **The Code:** Just standard anchors pointing to physical files on a server.

The **History API** is a set of methods and properties provided by the browser (part of the `window` object) that allows developers to manipulate the browser's session history—the list of pages visited in the current tab.

In modern web development, it is the engine that makes **Single Page Applications (SPAs)** possible by allowing the URL to change without a full page reload.

## 2. Routing via the History API

To create a smoother experience, developers started using the **HTML5 History API**. This allows JavaScript to change the URL in the address bar *without* triggering a page refresh.

- **`history.pushState()`**: Adds an entry to the browser's session history. The URL changes, but the browser stays on the same page.
- **`window.onpopstate`**: An event that fires when the user clicks the "Back" or "Forward" buttons, allowing the JS to update the UI accordingly.

### 3. Single Page Applications (SPA) & React Router

A **Single Page Application (SPA)** loads only one HTML file (usually `index.html`). When you "navigate," React simply hides one component and shows another, It loads CDN link only once and has smooth UI

**React Router DOM** is the standard library that manages this. It synchronizes your components with the URL using the History API under the hood. It ensures that if a user shares a link like `mysite.com/dashboard`, the app knows to render the Dashboard component immediately.

---

### 4. Key Components in React Router

To use React Router, you wrap your app in a hierarchy of components:

#### BrowserRouter

This is the parent component that stores the current location and helps navigate using the browser's history stack. You usually wrap your entire `<App />` in this.

#### Routes

Think of this as a **Switchboard**. It looks through all its child `Route` elements to find the one that matches the current URL and renders *only* that one.

#### Route

The individual mapping. It requires two main props:

- **path**: The URL segment (e.g., `/profile`).
  - **element**: The React component to show when the path matches.
-

## 5. Program Example

Here is how these pieces look in a real React application:

JavaScript

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
```

```
function App() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <Link to="/">Home</Link>  
        <Link to="/about">About</Link>  
      </nav>  
  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="*" element={<NotFound />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

---

## 6. Real-Life Analogy: The Magic Museum

To understand the difference, imagine a **Museum of History**.

- **Traditional Routing (The Old Way):** To see the "Ancient Egypt" exhibit, you have to leave the building, walk to a completely different building down the street, and go through security all over again.
- **SPA / React Router (The New Way):** You are in a **Magic Museum**. When you want to see "Ancient Egypt," you stay in the same room, but the walls rotate, the artifacts swap out, and the sign on the door instantly changes to say "Egypt Section." You never left the building; the environment just transformed around you.

| Feature        | Traditional Routing        | React Router (SPA)            |
|----------------|----------------------------|-------------------------------|
| Server Request | New request for every page | One request for the whole app |
| Page Load      | Full refresh (Blink)       | Instant component swap        |
| State          | Lost on navigation         | Preserved in memory           |
| Feel           | Multi-page document        | Native App-like experience    |